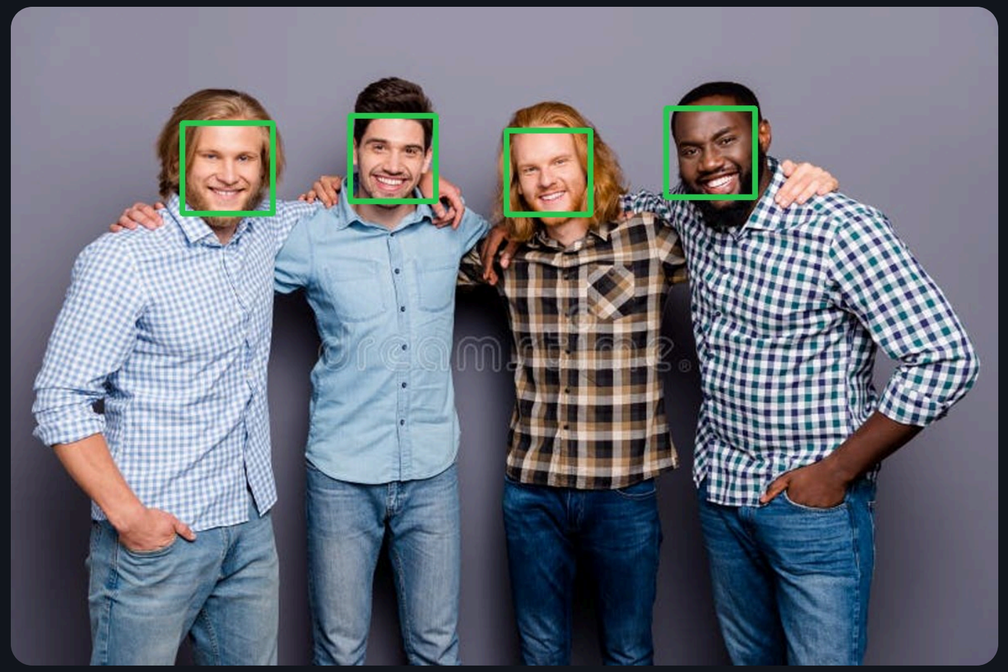
VISÃO COMPUTACIONAL · ATIVIDADE PRÁTICA

Reconhecimento de rostos

em imagens estáticas com Python e OpenCV

Márcio Augusto e Daniel Conceição



Como o classificador acha um rosto

01

Tons de cinza

A foto perde as cores. Para achar um rosto, só importa o brilho: o que é claro e o que é escuro.

02

Tipos de características

São as características de Haar: retângulos que comparam áreas claras e escuras. Os olhos são mais escuros que as bochechas.



03

Processo de classificação

Uma janela percorre a foto em vários tamanhos. Cada pedaço passa por uma cascata de etapas e é descartado na primeira que falhar.

Detalhe técnico: no OpenCV 5 o CascadeClassifier saiu do pacote principal. Por isso usamos a versão 4.x.

scaleFactor=1.1 · minNeighbors=5 (código original)

As 3 fotos testadas

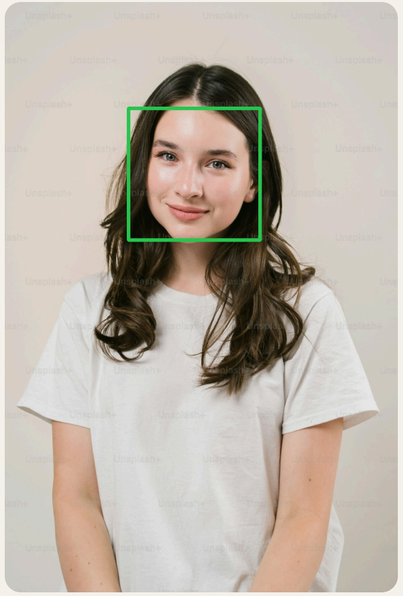


Foto 1 · uma pessoa

1 de 1

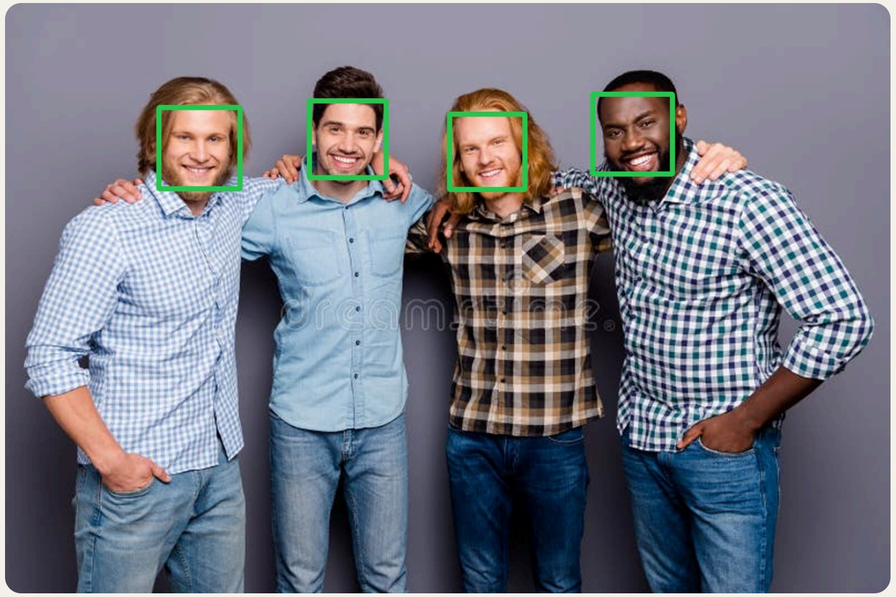


Foto 2 · grupo de 4 pessoas

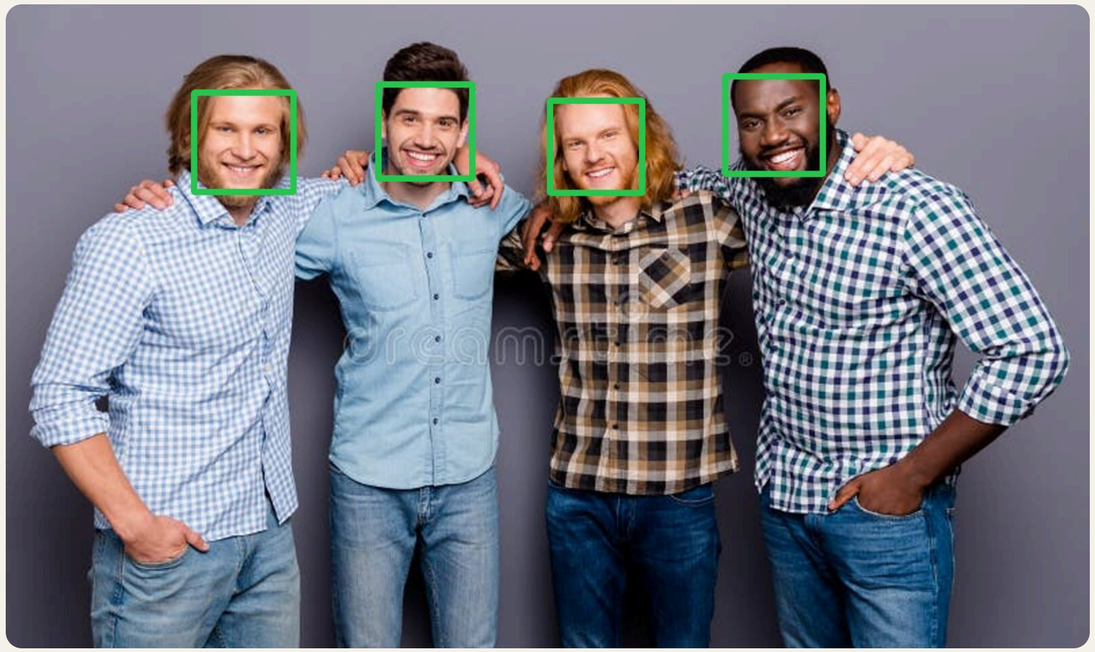
4 de 4



Foto 3 · multidão, cerca de 100 pessoas

0 de ~100 + 1 erro

Melhor e pior desempenho



Melhor: foto 2, 4 de 4

Rostos de frente, luz uniforme e fundo liso. Acertou tudo, sem nenhum erro, em 7 das 9 combinações de parâmetros.



Pior: foto 3, 0 de ~100

Rostos minúsculos, cabeças baixas, de lado e com óculos. Nenhum rosto certo em nenhuma das 9 combinações.

Por que a multidão derrotou o detector

Rostos pequenos demais

Cada rosto tem de 10 a 20 px. O modelo foi treinado com uma janela de 24 × 24 px: abaixo disso ele nem procura. Baixar o minSize não mudou nada.

Cabeça baixa ou de lado

O modelo só conhece rostos de frente. Quem está olhando o celular some.

Oclusão

Óculos escuros, bonés, cabelo e outras pessoas na frente.



Teste extra: ampliando a foto 3×, ele achou 21 rostos certos (verde) e 4 erros (laranja). Um dos erros eram óculos escuros no alto da cabeça.

9 combinações de parâmetros

scaleFactor	minNeighbors	Foto 1 · 1 rosto	Foto 2 · 4 rostos	Foto 3 · ~100 rostos
1.05	3	7 (6 falsos)	6 (2 falsos)	2 (2 falsos)
1.05	5	5 (4 falsos)	5 (1 falso)	2 (2 falsos)
1.05	8	3 (2 falsos)	4 (0 falsos)	1 (1 falso)
1.1	3	4 (3 falsos)	4 (0 falsos)	2 (2 falsos)
1.1	5 · original	1 (0 falsos)	4 (0 falsos)	1 (1 falso)
1.1	8	1 (0 falsos)	4 (0 falsos)	0
1.2	3	2 (1 falso)	4 (0 falsos)	1 (1 falso)
1.2	5	1 (0 falsos)	4 (0 falsos)	0
1.2	8	1 (0 falsos)	4 (0 falsos)	0

Cada célula: rostos detectados (quantos não eram rostos). Na foto 3, nenhuma combinação achou um rosto de verdade.

Padrão de classe e rigor

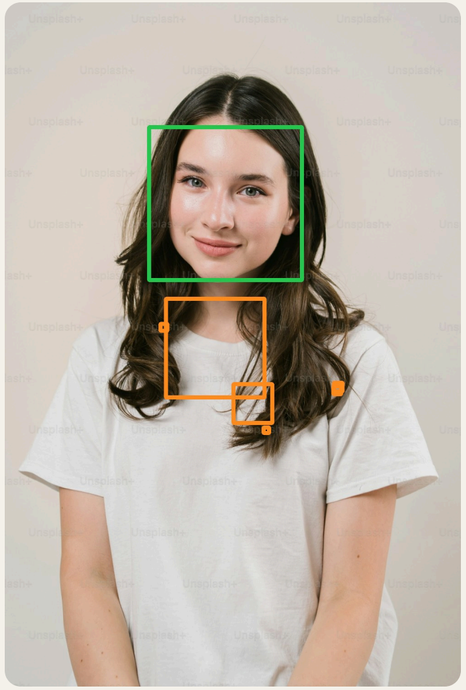


Foto 1 com 1.05 e 3: 7 marcações, 6 erradas (cabelo e camisa)

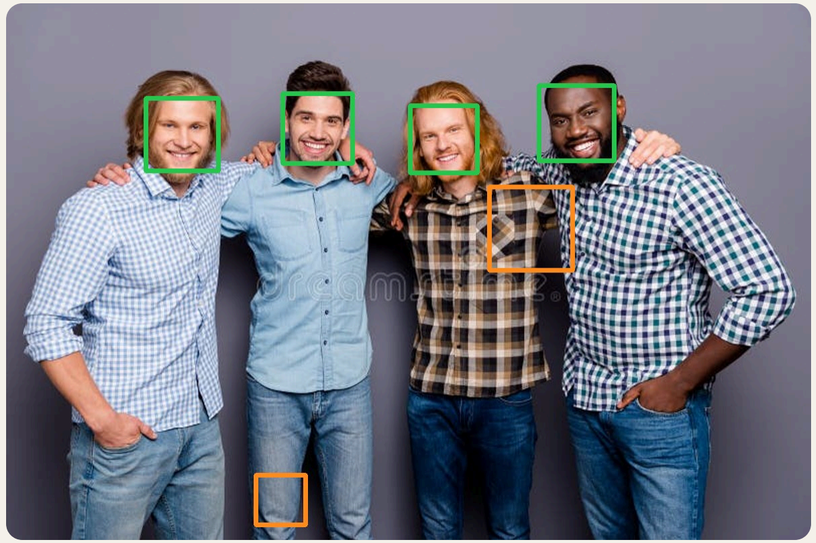


Foto 2 com 1.05 e 3: a camisa xadrez e a calça jeans viraram “rosto”. Xadrez é puro padrão claro/escuro.

O padrão de classe é o “rosto” que o modelo aprendeu no treino, só com rostos de frente. Os parâmetros definem o quanto uma região precisa se parecer com ele.

`minNeighbors ↑`

Mais rigoroso: menos rostos falsos, porém mais rostos perdidos.

`scaleFactor ↓`

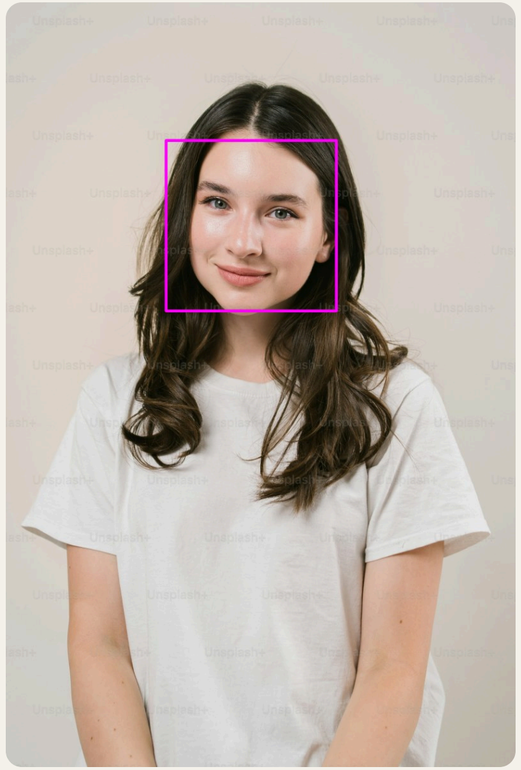
Testa mais tamanhos: mais sensível, mais lento e mais alarmes falsos.

Com 1.2 e 8, o porteiro ficou tão rígido que na multidão não entrou ninguém.

Mudança visual: cor e espessura



Antes: verde, espessura 2



Depois: magenta, espessura 15

A foto 1 tem 3000 px de largura. Quando ela é reduzida para 800 px, a linha de 2 px quase some. A espessura precisa acompanhar o tamanho da foto.

```
cv2.rectangle(img, p1, p2, (255, 0, 255), 15)
```

Cuidado: o OpenCV usa BGR, não RGB. **(255, 0, 255)** é magenta e **(0, 0, 255)** é vermelho, não azul.



APLICAÇÃO PRÁTICA

Borrar rostos antes de publicar

Pela LGPD, a imagem de uma pessoa é dado pessoal. Em fotos de eventos, escolas ou da rua, o detector acha os rostos e o script borra cada um sozinho.

```
cv2.GaussianBlur(rosto, (51, 51), 30)
```

Outras ideias: contar pessoas em eventos e validar foto 3×4 de documento (exatamente 1 rosto de frente).